Distributed Systems

Unit IV: Replication and Fault Tolerance

Syllabus:

Replication: Reasons for replication, Replica management – Finding the best server location, Content replication and placement, Content distribution, Managing replicated objects

Consistency protocols: Primary based protocols, replicated write protocols

Fault Tolerance: Introduction to fault tolerance, Reliable client server communication, Reliable group communication, distributed commit, Recovery – Check pointing, Message logging

Case Study: Caching and replication in web

- **Replication**

May_Jun_2024

Q3) a) Why replication is important and how it relates with Scalability? How are replicas kept consistent? Explain the mechanism of Server-Initiated Replicas with suitable example.

Ans. Importance of Replication

Replication means maintaining multiple copies of data or services on different machines in a distributed system.

Why Replication is Important

1. **Improves Availability**
    a. If one server fails, another replica can continue providing service.
    b. Ensures fault tolerance and high uptime.
2. **Enhances Reliability**
    a. Data is not lost even if hardware crashes occur.
    b. Backup copies increase system dependability.
3. **Increases Performance**
    a. Multiple replicas can serve requests simultaneously.
    b. Reduces response time by serving users from nearby servers.
4. **Supports Load Balancing**

    a. Requests are distributed among replicas.
    b. Prevents overload on a single server.
5. **Disaster Recovery**
    a. Replicas stored at different locations help recover from disasters.

Relation Between Replication and Scalability

Scalability is the ability of a system to handle increasing users, requests, or data efficiently.

Replication directly supports scalability because:

- More replicas ⇒ more servers available to process requests.
- Workload can be divided among replicas.
- Read operations especially become faster and more distributed.

Example

Suppose one web server can handle **1,000 users**.

- With 1 server → 1,000 users
- With 5 replicated servers → approximately 5,000 users

Thus replication enables **horizontal scaling** by adding more machines instead of upgrading one machine.

How Replicas are Kept Consistent

Since multiple copies of the same data exist, all replicas must contain updated and identical information.

This is called **Replica Consistency**.

Common Consistency Mechanisms

1. **Update Propagation**
    a. Whenever data changes, updates are sent to all replicas.
2. **Synchronous Replication**
    a. All replicas are updated before confirming the operation.
    b. Strong consistency but slower.
3. **Asynchronous Replication**
    a. Primary server updates replicas later.
    b. Faster but temporary inconsistencies may occur.
4. **Primary-Backup Protocol**
    a. One server acts as primary.
    b. All updates go through the primary first.

5. **Quorum-Based Methods**
    a. A minimum number of replicas must agree before reads/writes succeed.

Server-Initiated Replicas

In **server-initiated replication**, the **server decides** when and where replicas should be created, deleted, or updated.

Clients do not control replication.

The server monitors:

- workload,
- access frequency,
- network traffic,
- server load,

and automatically creates replicas when necessary.

Mechanism of Server-Initiated Replication

Working Steps

1. **Client Requests Data**
    a. Many users repeatedly access the same server.
2. **Server Detects Heavy Load**
    a. The server notices increasing requests or network congestion.
3. **Server Creates Replica**
    a. A new copy of data/service is created on another machine closer to users.
4. **Requests are Redirected**
    a. Clients are served from the nearest or least-loaded replica.
5. **Consistency Maintenance**
    a. Original server propagates updates to all replicas.

Suitable Example

Consider a video streaming platform like Netflix.

Scenario

A newly released movie becomes extremely popular in India.

Without Replication

- All users access a single U.S. server.
- High latency and server overload occur.

With Server-Initiated Replication

1. Central server detects huge traffic from India.
2. It automatically creates replicas in Indian data centers.
3. Indian users are redirected to nearby replicas.
4. Streaming becomes faster and scalable.
5. If the movie file changes, updates are propagated to all replicas.

Advantages of Server-Initiated Replication

- Automatic load balancing
- Better scalability
- Reduced response time
- Improved availability
- Efficient resource utilization

Disadvantages

- Complex consistency management
- Extra storage cost
- Synchronization overhead
- Replica placement decisions can be difficult

Conclusion

Replication is essential in distributed systems because it improves availability, reliability, performance, and scalability. Scalability is achieved by distributing workload among multiple replicas. To maintain correctness, replicas must remain consistent through synchronization techniques. In server-initiated replication, servers automatically create and manage replicas based on demand, making systems more efficient and scalable.

Nov_Dec_2024

Q3) b) With a neat diagram, explain the three types of replicas and their logical organization.

Ans. Types of Replicas and Their Logical Organization

In distributed systems, **replicas** are copies of data or services stored on multiple machines to improve:

- availability,
- reliability,

- fault tolerance,
- and performance.

Replicas are logically organized based on how they are managed and updated.

The three main types of replicas are:

1. **Permanent Replicas**
2. **Server-Initiated Replicas**
3. **Client-Initiated Replicas (Caches)**

1. Permanent Replicas

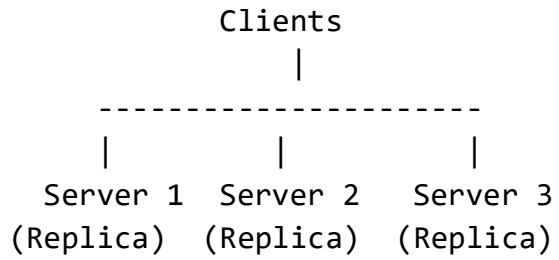
Permanent replicas are fixed copies of data stored on stable servers.

- Created manually or administratively.
- Usually remain active permanently.
- Commonly used in distributed databases and web services.

Characteristics

- Always available
- Highly reliable
- Located at predefined servers
- Managed by system administrators

Logical Organization of Permanent Replicas



All servers permanently store the same data.

Example

Distributed banking databases where branches maintain permanent copies of customer records.

2. Server-Initiated Replicas

In this approach, the server automatically creates replicas when demand increases.

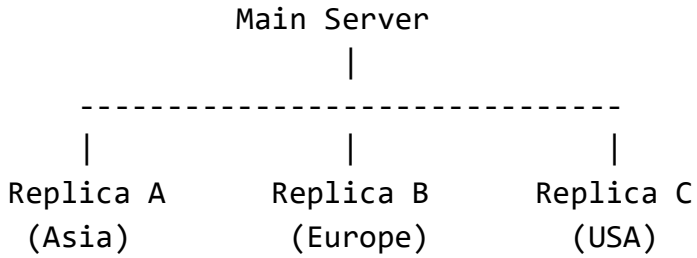
The server decides:

- when to create replicas,
- where to place them,
- and when to remove them.

Characteristics

- Dynamic creation
- Used for load balancing
- Improves scalability
- Reduces network traffic

Logical Organization of Server-Initiated Replicas



The main server distributes copies to different geographic locations.

Example

A streaming service like Netflix creates replicas in different countries when a movie becomes popular.

3. Client-Initiated Replicas (Caches)

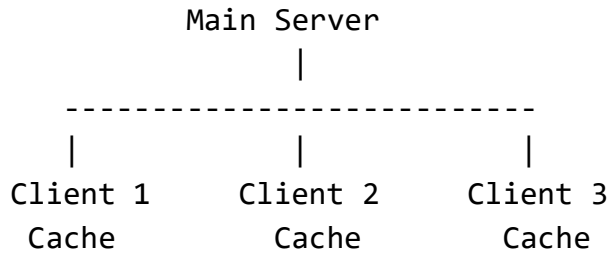
These replicas are created by clients temporarily.

Clients store frequently accessed data locally in cache memory.

Characteristics

- Temporary replicas
- Located near client
- Reduce server access
- Improve response time

Logical Organization of Client-Initiated Replicas



Each client keeps a local cached copy of data.

Example

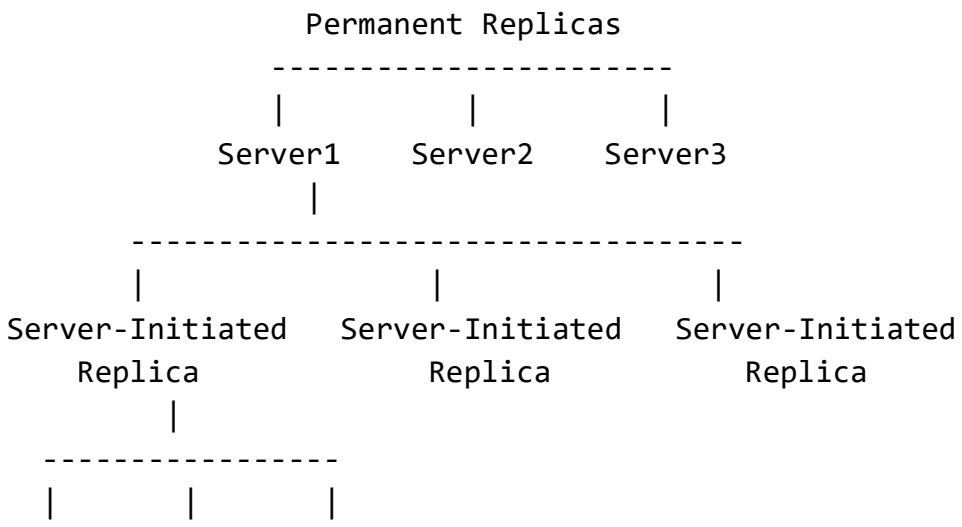
Web browsers storing web pages locally to reduce repeated downloads.

Examples include browsers from Google and Mozilla.

Comparison of Three Types of Replicas

Type	Created By	Lifetime	Purpose
Permanent Replica	Administrator/System	Long-term	Reliability & availability
Server-Initiated Replica	Server	Dynamic	Scalability & load balancing
Client-Initiated Replica	Client	Temporary	Faster access & reduced latency

Overall Logical Organization



```
Client  Client  Client
 Cache    Cache   Cache
```

This structure shows:

- permanent replicas at core servers,
- dynamic server replicas for scaling,
- and client caches for fast local access.

Conclusion

The three types of replicas—permanent replicas, server-initiated replicas, and client-initiated replicas—provide different advantages in distributed systems. Permanent replicas improve reliability, server-initiated replicas improve scalability and load balancing, while client-initiated replicas reduce latency and improve performance through caching. Together, they help distributed systems achieve high availability, scalability, and efficiency.

➢ Reasons for replication

May_Jun_2023

May_Jun_2025

Q4) b) What are two primary reasons for replication? Explain the causal consistency model with suitable example using distributed shared database.

Q3) a) Discuss the two important reasons for wanting to replicate data and how does replication relate to scalability.

Ans. Primary Reasons for Replication

Replication means storing multiple copies of data or services on different machines in a distributed system.

Two Primary Reasons for Replication

1. Reliability / Availability

Replication improves system reliability because if one server or node fails, another replica can continue providing the service.

Benefits

- Fault tolerance

- High availability
- Data recovery after failures
- Reduced risk of data loss

Example

If a banking database server crashes, another replicated server can continue handling transactions without interrupting customer services.

2. Performance Improvement

Replication improves performance by distributing workload among multiple servers.

Benefits

- Faster response time
- Reduced latency
- Load balancing
- Better handling of large numbers of users

Example

A website with replicas in different countries allows users to access the nearest server, reducing network delay.

Relation Between Replication and Scalability

Scalability

Scalability is the ability of a distributed system to handle increasing workload efficiently.

Replication supports scalability because:

- More replicas can serve more users simultaneously.
- Read requests can be distributed among replicas.
- System capacity increases by adding more machines.

Example

Suppose one server handles 2,000 requests per second.

- 1 server → 2,000 requests/sec
- 5 replicated servers → approximately 10,000 requests/sec

Thus, replication enables **horizontal scalability** by expanding the system using additional servers.

Causal Consistency Model

Causal consistency is a consistency model used in distributed systems where **causally related operations must be seen by all processes in the same order**.

However, operations that are unrelated may be seen in different orders on different machines.

Meaning of "Causally Related"

Operation B is causally dependent on operation A if:

- B is influenced by A, or
- B occurs because A happened first.

This follows the **cause-and-effect relationship**.

Rule of Causal Consistency

If:

- Write W1 causally affects Write W2,

then every process must observe:

- W1 before W2.

But independent writes can appear in different orders.

Example Using Distributed Shared Database

Consider a distributed social media database replicated across servers.

There are two users:

- User A
- User B

Shared database stores messages.

Step-by-Step Scenario

Step 1: User A Posts a Message

```
W1: "Project submission is tomorrow."
```

Step 2: User B Reads the Message and Replies

```
W2: "Thanks for the reminder!"
```

Here:

- W2 depends on W1.
- The reply exists because the original message was seen.

Therefore:

```
W1 → W2
```

(Causal relationship exists)

What Causal Consistency Ensures

Every replica in the distributed database must show:

```
"Project submission is tomorrow."
before
"Thanks for the reminder!"
```

No user should see the reply before seeing the original message.

Independent Operations

Suppose another user simultaneously posts:

```
W3: "Lunch break at 1 PM."
```

W3 is unrelated to W1 and W2.

Therefore:

- Some replicas may show W3 before W1.
- Others may show W1 before W3.

This is acceptable because no causal relationship exists.

Diagram Representation

```
User A                User B
W1: Original Message
        ↓
   Read W1
        ↓
W2: Reply Message
```

Causal order:

```
W1 → W2
```

All replicas must preserve this order.

Advantages of Causal Consistency

- Preserves logical cause-effect relationships
- More efficient than strict consistency
- Suitable for collaborative and social applications
- Allows higher concurrency

Disadvantages

- More complex implementation
- Tracking causal dependencies requires overhead
- Temporary inconsistencies may still occur

Applications

Causal consistency is commonly used in:

- Distributed shared databases
- Social media systems
- Collaborative editing systems
- Messaging applications
- Distributed file systems

Examples include systems developed by companies like Meta and Amazon for large-scale distributed services.

Conclusion

The two primary reasons for replication are improving reliability and enhancing performance. Replication also helps scalability by distributing workload among multiple servers. Causal consistency ensures that causally related operations are observed in the same order across all replicas, maintaining logical correctness while still allowing efficient distributed operation.

➢ Replica Management

May_Jun_2023

Q3) a) What are the key issues in Replica Management? Explain the following with respect to content replication and placement with suitable diagram.i) Permanent Replicas ii) Server-Initiated Replicas iii) Client-Initiated Replicas

Ans. Key Issues in Replica Management

Replica management is the process of controlling, maintaining, and coordinating multiple copies of data or services in a distributed system.

The main goal is to ensure:

- consistency,
- availability,
- performance,
- and fault tolerance.

However, managing replicas introduces several important challenges.

1. Replica Placement

Issue

Determining:

- where replicas should be stored,
- how many replicas should exist,
- and which servers should contain them.

Challenges

- Too many replicas increase storage cost.
- Too few replicas reduce availability and performance.
- Geographic placement affects latency.

Example

A global application may place replicas in:

- Asia,
- Europe,
- and North America

to serve users faster.

2. Consistency Management

Issue

Ensuring all replicas contain the same updated data.

When one replica changes, updates must propagate to others.

Challenges

- Network delays
- Concurrent updates
- Synchronization overhead

Types

- Strong consistency
- Weak consistency
- Eventual consistency
- Causal consistency

Example

In an online banking system, all replicas must show the same account balance after a transaction.

3. Update Propagation

Issue

How updates are distributed to replicas.

Challenges

- Fast propagation increases traffic.
- Delayed propagation may cause stale data.

Methods

Push-Based

Server sends updates automatically.

Pull-Based

Replicas periodically request updates.

4. Replica Synchronization

Issue

Keeping replicas synchronized during concurrent operations.

Challenges

- Conflicting writes

- Simultaneous updates
- Ordering of operations

Example

Two users editing the same document simultaneously.

5. Fault Tolerance and Recovery

Issue

Handling replica or server failures.

Challenges

- Detecting failed replicas
- Recovering lost data
- Restoring synchronization

Example

If one database replica crashes, another replica should immediately take over.

6. Scalability

Issue

Managing replicas efficiently as the system grows.

Challenges

- More replicas increase coordination overhead.
- Maintaining consistency becomes difficult at large scale.

Example

Large cloud systems may maintain thousands of replicas worldwide.

Companies like Amazon and Google face this challenge in global distributed systems.

7. Transparency

Issue

Hiding replication details from users.

Users should feel they are accessing a single system even though multiple replicas exist.

Types of Transparency

- Access transparency
- Location transparency
- Replication transparency

8. Load Balancing

Issue

Distributing client requests evenly among replicas.

Challenges

- Preventing server overload
- Efficient request routing

Example

Web servers distribute traffic across multiple replicated servers.

9. Security

Issue

Protecting replicated data from unauthorized access or attacks.

Challenges

- Secure synchronization
- Authentication among replicas
- Preventing data tampering

10. Storage and Cost Overhead

Issue

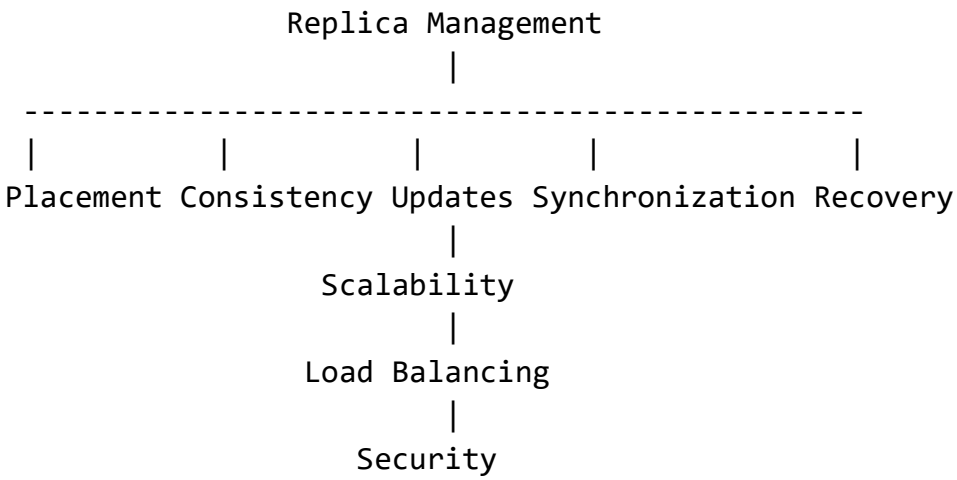
Replication increases:

- storage requirements,
- bandwidth usage,
- and maintenance cost.

Challenges

Balancing performance benefits with infrastructure cost.

Diagram of Replica Management Issues



Summary Table

Issue	Description
Replica Placement	Deciding where replicas are stored
Consistency	Keeping all copies identical
Update Propagation	Distributing updates efficiently
Synchronization	Coordinating concurrent operations
Fault Tolerance	Recovering from failures
Scalability	Handling growth efficiently
Transparency	Hiding replication details
Load Balancing	Distributing workload
Security	Protecting replicated data
Cost Overhead	Managing storage and network cost

Conclusion

Replica management is a critical part of distributed systems. It involves handling placement, consistency, synchronization, scalability, fault tolerance, and security of replicas. Efficient replica management ensures high availability, better performance, and reliable operation of distributed applications.

Replica Management in Distributed Systems

Replica management is the process of controlling and coordinating multiple copies (replicas) of data or services in a distributed system.

Its main objectives are:

- high availability,
- fault tolerance,
- scalability,
- reduced latency,
- and better performance.

Replica management mainly involves:

1. Finding the Best Server Location
2. Content Replication and Placement
3. Content Distribution
4. Managing Replicated Objects

1. Finding the Best Server Location

Definition

When multiple replicas exist, the system must determine which server is best suited to handle a client request.

The aim is to:

- minimize response time,
- reduce network delay,
- balance load,
- and improve user experience.

Factors Used to Select Best Server

1. Geographic Distance

Nearest server is usually preferred.

2. Network Latency

Server with minimum communication delay is selected.

3. Server Load

Less-loaded server gives better performance.

4. Availability

Only active and reachable servers are selected.

5. Data Freshness

Latest updated replica may be preferred.

Techniques

DNS-Based Selection

DNS redirects client to nearest server.

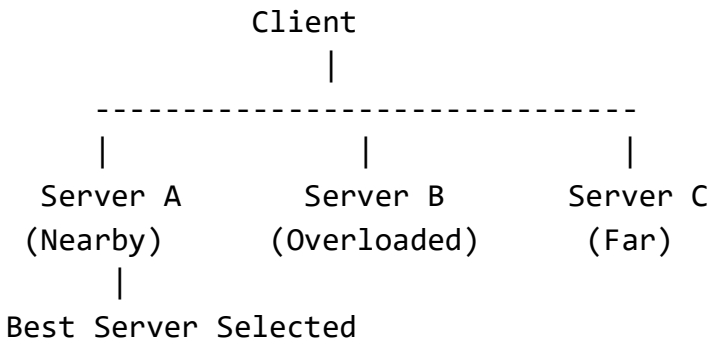
Load Balancers

Requests distributed among servers.

Anycast Routing

Network automatically chooses closest server.

Diagram



Example

Streaming platforms like Netflix route users to nearby servers for smooth video playback.

2. Content Replication and Placement

Content Replication

Definition

Creating multiple copies of content/data across different servers.

Purpose

- availability,
- reliability,
- scalability,
- fault tolerance.

Content Placement

Definition

Determining:

- where replicas should be placed,
- how many replicas should exist,
- and when replicas should move or be removed.

Placement Strategies

Static Placement

Fixed replica locations.

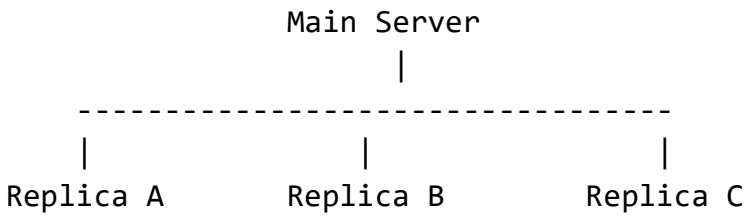
Dynamic Placement

Replicas created automatically according to demand.

Geographic Placement

Replicas distributed globally.

Diagram



```
(Asia)          (Europe)          (USA)
```

Example

Popular videos are replicated to regional data centers to reduce buffering.

Advantages

- Faster access
- Better scalability
- Reduced network traffic
- Improved reliability

Challenges

- Consistency maintenance
- Storage overhead
- Synchronization cost

3. Content Distribution

Definition

Content distribution is the efficient delivery of data and services from servers to users over a distributed network.

Goals

- low latency,
- high performance,
- scalability,
- high availability.

Components

Origin Server

Stores original content.

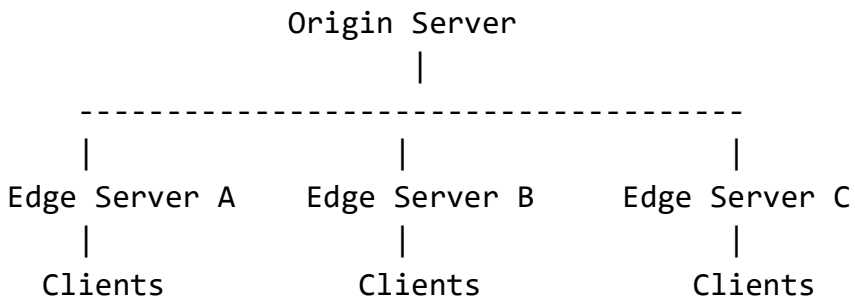
Edge Servers / Replica Servers

Store replicated content near users.

Clients

Request content.

Diagram



Techniques Used

Replication

Multiple copies of content.

Caching

Temporary local storage.

CDN (Content Delivery Network)

Distributed edge servers deliver content efficiently.

Example

CDNs provided by Cloudflare distribute website and media content globally.

Advantages

- Reduced latency
- Faster content delivery
- Better load balancing
- Fault tolerance

Challenges

- Consistency management
- Bandwidth cost
- Security issues

4. Managing Replicated Objects

Definition

Managing replicated objects means maintaining consistency and coordination among multiple copies of shared data objects.

Main Issues in Managing Replicated Objects

1. Consistency Management

All replicas should contain correct and updated data.

Types

- Strong consistency
- Weak consistency
- Eventual consistency
- Causal consistency

2. Update Propagation

Changes made to one replica must be propagated to others.

Methods

- Push-based
- Pull-based

3. Synchronization

Concurrent operations must be coordinated properly.

4. Replica Control

Managing:

- creation,
- deletion,
- migration,
- recovery of replicas.

5. Fault Handling

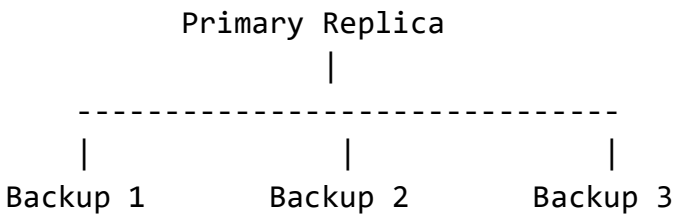
Failed replicas must be detected and restored.

Primary-Based Protocol

One server acts as primary:

- all writes go through primary,
- updates sent to backup replicas.

Diagram



Example

Distributed databases use replicated object management to maintain identical data copies across multiple servers.

Companies like Amazon and Google use advanced replica management in cloud systems.

Advantages of Replica Management

- High availability
- Fault tolerance
- Better performance
- Scalability
- Reduced latency
- Load balancing

Challenges of Replica Management

Challenge	Description
Consistency	Keeping replicas synchronized
Storage Cost	More replicas require more space
Network Overhead	Update propagation consumes bandwidth
Complexity	Managing many replicas is difficult
Fault Recovery	Failed replicas must recover quickly

Conclusion

Replica management is a vital part of distributed systems. It involves selecting the best server, replicating and placing content efficiently, distributing content to users, and

managing replicated objects consistently. Proper replica management improves performance, scalability, reliability, and availability in modern distributed applications.

Nov_Dec_2024

Q4) b) Discuss and compare “Push versus Pull Protocols” propagation design issue for content distribution

Ans. Push versus Pull Protocols in Content Distribution

In distributed systems, replicated data or content must remain updated across multiple servers or replicas.

The process of sending updates from one replica to others is called **update propagation**.

Two major propagation design approaches are:

1. **Push Protocol**
2. **Pull Protocol**

These protocols are widely used in:

- content distribution systems,
- replicated databases,
- caching systems,
- CDNs,
- and distributed file systems.

1. Push Protocol

Definition

In a push protocol, the **server actively sends updates** to replicas or clients whenever content changes.

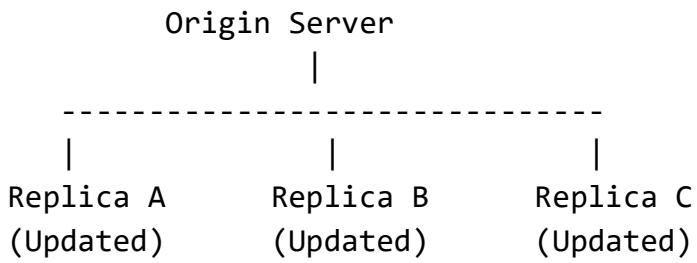
The receiver does not request updates.

Working of Push Protocol

Steps

1. Content is updated at the origin server.
2. Server immediately propagates updates to replicas.
3. Replicas receive and apply updates automatically.

Diagram of Push Protocol



Updates are pushed automatically to all replicas.

Characteristics

- Server-driven communication
- Immediate update propagation
- Better consistency
- Higher bandwidth usage

Example

A news website instantly pushes breaking news updates to all regional servers.

Streaming platforms like Netflix may push newly released content to edge servers before demand rises.

Advantages of Push Protocol

1. Fast Update Distribution

Replicas receive updates immediately.

2. Better Consistency

All replicas remain closely synchronized.

3. Reduced Stale Data

Clients access fresher content.

4. Suitable for Frequently Changing Data

Ideal when updates are frequent and important.

Disadvantages of Push Protocol

1. High Network Traffic

Updates sent even if replicas do not need them.

2. Wasted Bandwidth

Unnecessary updates may occur.

3. Scalability Issues

Large numbers of replicas increase propagation cost.

4. Server Overhead

Origin server manages update distribution.

2. Pull Protocol

Definition

In a pull protocol, replicas or clients **request updates periodically** from the server.

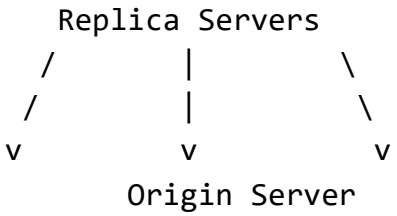
The server sends updates only when requested.

Working of Pull Protocol

Steps

1. Replica periodically checks server for updates.
2. Server responds with latest content if changes exist.
3. Replica updates local copy.

Diagram of Pull Protocol



Replicas pull updates from the server when needed.

Characteristics

- Client/replica-driven communication
- Periodic update requests

- Lower server workload
- Possible stale data

Example

Web browsers periodically refresh cached web pages.

Applications from Google often use pull-based cache validation mechanisms.

Advantages of Pull Protocol

1. Lower Network Usage

Updates sent only when requested.

2. Better Scalability

Server handles fewer unnecessary transmissions.

3. Efficient for Rare Updates

Suitable when data changes infrequently.

4. Reduced Server Load

Replicas control update timing.

Disadvantages of Pull Protocol

1. Stale Data Problem

Replicas may temporarily hold outdated content.

2. Delayed Consistency

Updates not immediately propagated.

3. Periodic Polling Overhead

Frequent checking can still waste resources.

Comparison Between Push and Pull Protocols

Feature	Push Protocol	Pull Protocol
Update Initiation	Server initiates	Replica/client initiates
Update Speed	Immediate	Periodic/delayed
Consistency	Stronger	Weaker
Network Usage	Higher	Lower
Scalability	Less scalable	More scalable
Server Load	High	Lower
Stale Data	Minimal	Possible
Suitable For	Frequently updated data	Rarely updated data

Hybrid Push-Pull Protocol

Many modern distributed systems use a combination of both protocols.

Working

- Important updates are pushed immediately.
- Other updates are pulled periodically.

Example

Content Delivery Networks (CDNs) from Cloudflare often use hybrid mechanisms:

- popular content is pushed,
- less-used content is pulled on demand.

Real-Life Analogy

Push Protocol

Teacher directly announces assignment updates to all students.

Pull Protocol

Students periodically check notice board for updates.

Conclusion

Push and pull protocols are important propagation mechanisms in content distribution systems. Push protocols provide faster consistency by actively distributing updates, while pull protocols reduce communication overhead by allowing replicas to request updates when needed. Modern distributed systems often combine both approaches to achieve better scalability, efficiency, and consistency.

- **Consistency Protocols**

May_Jun_2023

Q3) b) What is a primary-based protocol in a consistency protocol? Explain the working of replicated write protocols with active replication.

Ans. Primary-Based Protocol in Consistency Protocol

A **primary-based protocol** is a replica consistency protocol in which one replica is designated as the **primary replica (master)** and all other replicas act as **backup replicas (slaves)**.

The primary replica is responsible for:

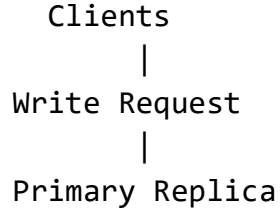
- handling write operations,
- maintaining update order,
- and propagating updates to backup replicas.

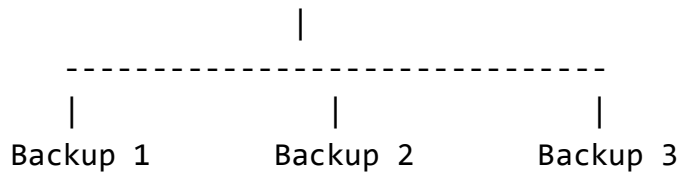
This protocol ensures that all replicas remain consistent.

Basic Idea

- Only the **primary server** is allowed to process write requests.
- Backup replicas cannot update data directly.
- Clients send update requests to the primary replica.
- Primary updates its copy first and then propagates changes to backups.

Diagram of Primary-Based Protocol





Working of Primary-Based Protocol

Step 1: Client Sends Write Request

Client sends update request to the primary server.

Step 2: Primary Updates Data

Primary replica performs the write operation.

Step 3: Update Propagation

Primary sends updated data to backup replicas.

Step 4: Acknowledgment

Backups confirm update reception.

Step 5: Client Notification

Primary informs client that write operation is complete.

Advantages

- Simple consistency management
- Easy ordering of updates
- Strong consistency possible

Disadvantages

- Primary server becomes bottleneck
- Single point of failure
- High load on primary replica

Replicated Write Protocols with Active Replication

Active Replication

In **active replication**, all replicas process the same operations in the same order.

Every replica:

- receives every write request,
- executes the operation independently,
- and reaches the same final state.

There is no separate backup execution.

Key Principle

All replicas are active simultaneously.

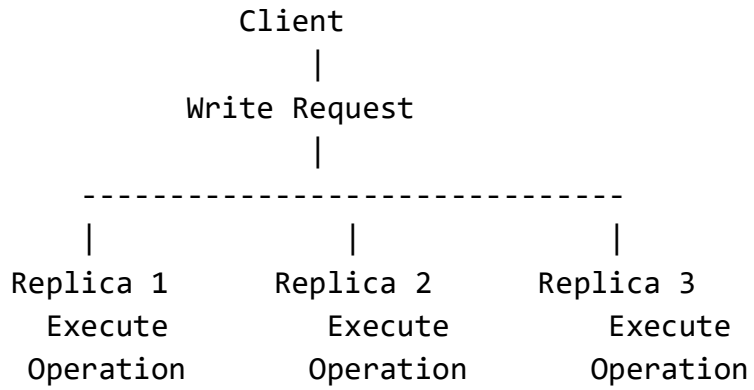
To maintain consistency:

- operations must be executed in identical order at every replica.

This is often achieved using:

- atomic broadcast,
- total ordering,
- or reliable multicast.

Diagram of Active Replication



All replicas perform the same write operation.

Working of Replicated Write Protocol with Active Replication

Step 1: Client Sends Write Request

Client sends operation request to the replica group.

Example:

```
Deposit ₹1000
```

Step 2: Request Ordering

The system ensures all replicas receive requests in the same order.

Example order:

```
Request 1 → Request 2 → Request 3
```

Step 3: All Replicas Execute Operation

Each replica independently performs the same operation.

Example:

- Replica 1 updates balance
- Replica 2 updates balance
- Replica 3 updates balance

All produce identical results.

Step 4: Response to Client

One replica or all replicas may send acknowledgment to client.

Example Using Distributed Bank Database

Suppose account balance is:

```
₹5000
```

Client requests:

```
Withdraw ₹1000
```

All replicas execute:

```
₹5000 - ₹1000 = ₹4000
```

Final balance at every replica becomes:

```
₹4000
```

Consistency is maintained because all replicas processed identical operations in same order.

Characteristics of Active Replication

Feature	Description
All replicas active	Every replica processes requests
Same execution order	Operations executed identically
High fault tolerance	Failure of one replica does not stop system
Fast recovery	Other replicas already updated

Advantages of Active Replication

- High availability
- Excellent fault tolerance
- Fast failover
- No recovery delay
- Strong consistency possible

Disadvantages

- High communication overhead
- Complex ordering mechanisms
- Increased processing cost
- Difficult synchronization

Difference Between Primary-Based and Active Replication

Feature	Primary-Based	Active Replication
Write Handling	Only primary writes	All replicas write
Replica Role	Primary + backups	All replicas equal
Complexity	Simpler	More complex
Fault Tolerance	Moderate	High
Performance Overhead	Lower	Higher

Applications

Active replication is commonly used in:

- fault-tolerant systems,
- distributed databases,
- air traffic systems,
- banking systems,
- cloud services.

Used in large-scale systems developed by Google and Microsoft.

Conclusion

A primary-based protocol maintains consistency by allowing only one primary replica to process write operations and propagate updates to backups. In active replication, all replicas execute the same write operations in identical order, ensuring strong consistency and high fault tolerance. Both approaches are widely used consistency protocols in distributed systems.

Nov_Dec_2023

Q3) b) What is a primary-based protocol in a consistency protocol? Explain the working of primary-backup protocol with suitable diagram.

Ans. Primary-Backup Protocol

The **Primary-Backup Protocol** is a replica consistency protocol used in distributed systems to maintain consistency among replicated data.

In this protocol:

- one replica acts as the **Primary (Master)**,
- other replicas act as **Backups (Slaves)**.

The primary server handles all write operations and propagates updates to backup replicas.

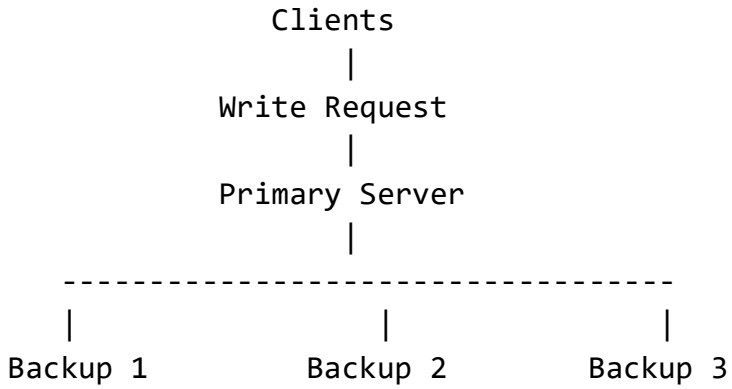
This protocol ensures:

- consistency,
- fault tolerance,
- reliability,
- and high availability.

Basic Concept

- Clients communicate with the primary server for updates.
- The primary performs the operation first.
- Updated data is then sent to backup replicas.
- Backups maintain identical copies of data.

Architecture of Primary-Backup Protocol



Working of Primary-Backup Protocol

Step 1: Client Sends Request

A client sends a read/write request to the primary server.

Example

```
Update Account Balance
```

Step 2: Primary Processes the Request

The primary replica:

- validates request,
- performs operation,
- updates its local copy.

Step 3: Propagation to Backups

The primary sends updated information to all backup replicas.

```
Primary → Backup Replicas
```

Step 4: Backups Update Their Copies

Each backup replica:

- receives update,
- modifies local data,
- sends acknowledgment to primary.

Step 5: Primary Sends Response to Client

After successful update propagation:

- primary confirms completion to client.

Read Operation

For read requests:

- client may contact primary,
- or sometimes read from nearest backup replica.

Example Using Banking System

Suppose account balance is:

`₹10,000`

Client requests:

`Deposit ₹2,000`

Execution Flow

Primary Updates Balance

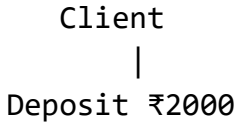
`₹10,000 + ₹2,000 = ₹12,000`

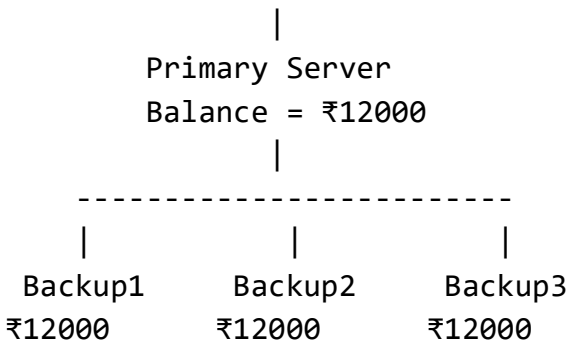
Backups Receive Update

All backup replicas update their balances to:

`₹12,000`

Diagram Showing Complete Flow





Types of Primary-Backup Protocols

1. Remote-Write Protocol

- All operations performed at primary.
- Backups only store updates.

2. Local-Write Protocol

- Migrating primary role possible.
- Write operations may occur locally after ownership transfer.

Advantages of Primary-Backup Protocol

1. Strong Consistency

All updates controlled centrally.

2. Simpler Management

Easy ordering of operations.

3. Fault Tolerance

Backups maintain data copies.

4. High Availability

Backup can replace failed primary.

Disadvantages

1. Primary Bottleneck

All writes pass through one server.

2. Single Point of Failure

Primary failure affects system temporarily.

3. Communication Overhead

Updates must propagate to backups.

4. Scalability Limitations

Large systems increase synchronization cost.

Failure Handling

If primary fails:

1. One backup is selected as new primary.
2. System redirects clients to new primary.
3. Operations continue.

This process may involve election algorithms.

Applications

Primary-backup protocols are used in:

- distributed databases,
- banking systems,
- cloud storage,
- replicated file systems,
- transaction systems.

Used in systems developed by Microsoft and Oracle.

Conclusion

The primary-backup protocol is a widely used consistency mechanism in distributed systems where one primary replica controls updates and backup replicas maintain synchronized copies. It provides strong consistency, fault tolerance, and reliability, making it suitable for critical distributed applications.

Nov_Dec_2024

Q3) a) Describe the following independent axes for defining inconsistencies with examples: i) Deviation in numerical values between replicas. ii) Deviation in staleness between replicas. iii) Deviation with respect to ordering of update operations.

Ans. Independent Axes for Defining Inconsistencies

In distributed systems, replicas may temporarily become inconsistent because updates are not received simultaneously by all replicas.

To measure these inconsistencies, three important independent axes are used:

1. Deviation in numerical values between replicas
2. Deviation in staleness between replicas
3. Deviation with respect to ordering of update operations

These axes describe different ways replicas may differ from each other.

i) Deviation in Numerical Values Between Replicas

Definition

This inconsistency measures the difference in the actual data values stored at different replicas.

It shows:

- how much one replica's value differs from another.

Explanation

When updates are delayed or missed, replicas may temporarily contain different numerical values.

The inconsistency is measured numerically.

Example

Consider a distributed banking system.

Correct account balance:

`₹50,000`

Replicas contain:

Replica	Balance
Replica A	₹50,000
Replica B	₹49,500
Replica C	₹50,000

Numerical Deviation

Replica B differs by:

₹500

Therefore:

- deviation in numerical value = ₹500.

Diagram

```
Distributed Account Database

Replica A → ₹50,000
Replica B → ₹49,500
Replica C → ₹50,000
```

Replica B contains inconsistent numerical data.

Applications

Important in:

- banking systems,
- stock markets,
- inventory systems.

ii) Deviation in Staleness Between Replicas

Definition

Staleness measures how outdated a replica is compared to the latest version of data.

It represents:

- delay in receiving updates,
- or number of missed updates.

Explanation

A stale replica has not yet received the latest changes.

Staleness may be measured by:

- time delay,
- timestamps,
- version numbers.

Example

Suppose latest weather data is updated at:

```
12:00 PM
```

Replica A receives update immediately.

Replica B receives update at:

```
12:10 PM
```

Staleness

Replica B is stale by:

```
10 minutes
```

Diagram

```
         Latest Update → 12:00 PM

      Replica A → Updated at 12:00 PM
      Replica B → Updated at 12:10 PM
```

Replica B contains stale information.

Applications

Used in:

- caching systems,
- DNS servers,
- CDNs,
- distributed file systems.

Companies like Cloudflare handle staleness carefully in global caching networks.

iii) Deviation with Respect to Ordering of Update Operations

Definition

This inconsistency occurs when replicas execute update operations in different orders.

Different ordering may produce different final results.

Explanation

In distributed systems:

- updates may occur concurrently,

- messages may arrive at different times,
- replicas may process operations in different sequences.

Example

Initial account balance:

```
₹100
```

Two operations occur:

```
U1: Add ₹50
U2: Multiply balance by 2
```

Replica A Executes

```
Step 1: ₹100 + ₹50 = ₹150
Step 2: ₹150 × 2 = ₹300
```

Final value:

```
₹300
```

Replica B Executes

```
Step 1: ₹100 × 2 = ₹200
Step 2: ₹200 + ₹50 = ₹250
```

Final value:

```
₹250
```

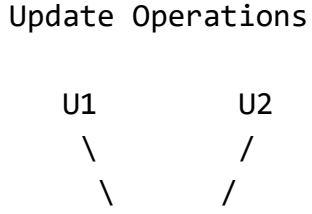
Problem

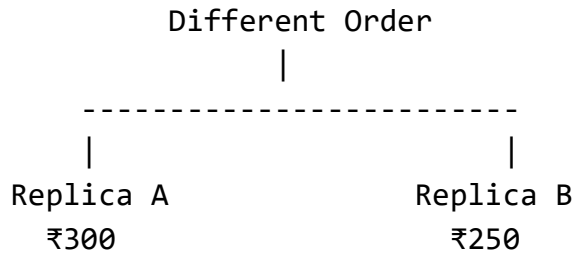
Different operation ordering produced:

- Replica A → ₹300
- Replica B → ₹250

This is ordering inconsistency.

Diagram





Applications

Critical in:

- distributed databases,
- transaction systems,
- collaborative editing,
- distributed shared memory.

Comparison Table

Axis	Meaning	Example
Numerical Deviation	Difference in actual values	₹50,000 vs ₹49,500
Staleness Deviation	Delay in updates	10-minute-old data
Ordering Deviation	Different order of operations	₹300 vs ₹250

Conclusion

The three independent axes of inconsistencies help analyze how replicas differ in distributed systems. Numerical deviation measures differences in stored values, staleness measures how outdated replicas are, and ordering deviation measures inconsistencies caused by different update sequences. These concepts are essential for designing reliable and efficient replica consistency models.

May_Jun_2025

Q4) b) Define data-centric consistency model. Explain the causal consistency model with suitable example using distributed shared database.

Ans. Data-Centric Consistency Model

Definition

A **data-centric consistency model** specifies the rules that define how updates to shared data are observed by all processes in a distributed system.

It ensures that:

- all users/processes see a consistent view of replicated data,
- even when multiple replicas exist on different machines.

The model describes:

- when updates become visible,
- in what order they are seen,
- and how replicas maintain consistency.

Purpose of Data-Centric Consistency Models

These models help:

- maintain correctness of shared data,
- coordinate concurrent access,
- and manage replicated databases.

They are mainly used in:

- distributed databases,
- distributed shared memory,
- cloud systems,
- replicated file systems.

Common Data-Centric Consistency Models

1. Strict Consistency
2. Sequential Consistency
3. Causal Consistency
4. FIFO Consistency
5. Weak Consistency
6. Eventual Consistency

Causal Consistency Model

Definition

The **causal consistency model** ensures that all processes observe **causally related operations in the same order**.

However, operations that are not causally related may be seen in different orders on different replicas.

Meaning of Causal Relationship

An operation is causally related to another if:

- one operation influences the other,
- or one operation happens because of another.

This follows a **cause-and-effect relationship**.

Rule of Causal Consistency

If:

```
Operation A → causes → Operation B
```

then every process must observe:

```
A before B
```

But unrelated operations may appear in different orders.

Example Using Distributed Shared Database

Consider a distributed social media database replicated across multiple servers.

There are two users:

- User A
- User B

Shared database stores posts and replies.

Step-by-Step Scenario

Step 1: User A Creates a Post

User A writes:

```
W1: "Meeting starts at 10 AM."
```

This update is stored in the distributed shared database.

Step 2: User B Reads the Post and Replies

After seeing W1, User B writes:

```
W2: "Okay, I will join on time."
```

Causal Relationship

The reply exists because User B first saw W1.

Therefore:

```
W1 → W2
```

W2 is causally dependent on W1.

What Causal Consistency Guarantees

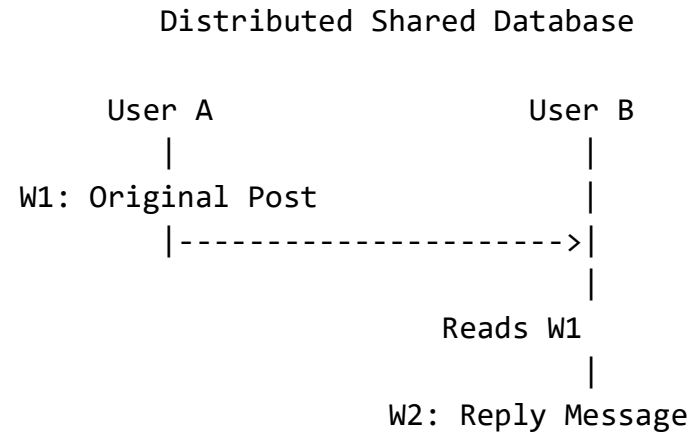
Every replica in the distributed database must display:

```
"Meeting starts at 10 AM."
before
"Okay, I will join on time."
```

No process should see:

```
reply before original message
```

Diagram



Causal order:

```
W1 → W2
```

All replicas must preserve this order.

Independent Operations

Suppose another user simultaneously posts:

```
W3: "Lunch break at 1 PM."
```

W3 is unrelated to W1 and W2.

Therefore:

- some replicas may see W3 before W1,
- others may see W1 before W3.

This is allowed because no causal dependency exists.

Characteristics of Causal Consistency

Feature	Description
Preserves causality	Cause-effect order maintained
Allows concurrency	Independent operations may vary
More flexible	Less strict than sequential consistency
Better performance	Reduced synchronization overhead

Advantages

- Maintains logical correctness
- Supports concurrent operations
- More efficient than strict consistency
- Suitable for collaborative applications

Disadvantages

- Complex dependency tracking
- Extra communication overhead
- Temporary inconsistencies may still exist

Applications

Causal consistency is widely used in:

- social media systems,
- collaborative editing,
- distributed databases,
- messaging applications.

Large-scale systems by Meta and Amazon use concepts related to causal consistency.

Conclusion

A data-centric consistency model defines how shared replicated data remains consistent in distributed systems. The causal consistency model specifically guarantees that causally related operations are observed in the same order by all processes, while unrelated operations may appear in different orders. This provides a balance between consistency and performance in distributed shared databases.

♦ **Fault Tolerance**

➢ **Introduction To Fault Tolerance**

May_Jun_2024

Q3) b) What is fault tolerance? Explain the transient, intermittent and permanent fault classes. Explain in brief, various types of failures.

Ans. Fault Tolerance

Definition

Fault tolerance is the ability of a distributed system to continue functioning correctly even when some components fail.

A fault-tolerant system:

- detects faults,
- isolates faulty components,
- recovers from failures,
- and continues providing services with minimal interruption.

Fault tolerance improves:

- reliability,
- availability,
- dependability,

- and system stability.

Need for Fault Tolerance

Distributed systems consist of:

- multiple computers,
- networks,
- databases,
- and communication links.

Failures may occur because of:

- hardware crashes,
- software bugs,
- network problems,
- power failures.

Without fault tolerance, a single failure may stop the entire system.

Fault Classes

Faults are commonly classified into:

1. Transient Faults
2. Intermittent Faults
3. Permanent Faults

1. Transient Fault

Definition

A transient fault occurs temporarily and disappears automatically after a short time.

The fault:

- appears once,
- affects the system briefly,
- then disappears without repair.

Characteristics

- Temporary in nature
- Difficult to predict
- May not reoccur
- Often caused by environmental conditions

Causes

- Electrical noise
- Temporary network congestion
- Cosmic radiation
- Sudden voltage fluctuation

Example

A network packet is corrupted once due to electrical interference, but later communication works normally.

Diagram

```
Time →
-----------------------------
 Fault Appears   Fault Gone
```

2. Intermittent Fault

Definition

An intermittent fault repeatedly appears and disappears over time.

The system works correctly sometimes and fails at other times.

Characteristics

- Occurs irregularly
- Difficult to diagnose
- Repeats periodically
- Unstable behavior

Causes

- Loose hardware connections
- Unstable network links
- Overheating components
- Software timing errors

Example

A server disconnects randomly due to a faulty network cable.

Diagram

```
Time →
-------------------------------------
 Fault   Normal   Fault   Normal
```

3. Permanent Fault

Definition

A permanent fault remains in the system until the faulty component is repaired or replaced.

The fault does not disappear automatically.

Characteristics

- Continuous failure
- Requires repair/replacement
- Easy to detect compared to intermittent faults

Causes

- Hardware damage
- Disk crash
- Burnt processor
- Software corruption

Example

A hard disk permanently stops functioning after physical damage.

Diagram

```
Time →
--------------------------------
 Fault Starts → Continues Forever
```

Comparison of Fault Classes

Fault Type	Nature	Duration	Example
Transient	Temporary	Short	Packet corruption
Intermittent	Repeatedly appears/disappears	Irregular	Loose connection
Permanent	Continuous	Long-term	Hard disk failure

Types of Failures in Distributed Systems

A failure occurs when a system cannot perform its intended function.

Main types of failures are:

1. Crash Failure
2. Omission Failure
3. Timing Failure
4. Response Failure
5. Byzantine Failure

1. Crash Failure

Definition

The server or process stops functioning completely.

After crashing:

- no further responses are produced.

Example

A server shuts down because of power failure.

Diagram

```
System Running → Crash → No Response
```

2. Omission Failure

Definition

A system fails to send or receive messages.

Types

Send Omission

Message not sent.

Receive Omission

Message not received.

Example

A router drops packets because of network congestion.

3. Timing Failure

Definition

A response occurs outside the expected time interval.

Example

A real-time traffic control system responds too late.

4. Response Failure

Definition

The system produces an incorrect response.

Types

Value Failure

Wrong output value.

State Transition Failure

Wrong sequence of operations.

Example

ATM deducts incorrect amount from account.

5. Byzantine Failure

Definition

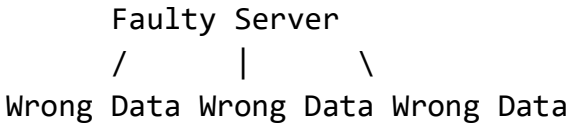
The system behaves unpredictably and may produce arbitrary or conflicting results.

This is the most difficult failure type to handle.

Example

A faulty server sends different data to different clients.

Diagram



Summary Table of Failure Types

Failure Type	Description
Crash Failure	System stops completely
Omission Failure	Messages lost
Timing Failure	Late/early response
Response Failure	Incorrect response
Byzantine Failure	Arbitrary unpredictable behavior

Fault Tolerance Techniques

Distributed systems achieve fault tolerance using:

- replication,
- redundancy,
- checkpointing,
- rollback recovery,
- failure detection,
- backup servers.

Companies like Google and Amazon use extensive fault-tolerant mechanisms in cloud systems.

Conclusion

Fault tolerance enables distributed systems to continue operating even when faults occur. Faults may be transient, intermittent, or permanent depending on their duration and behavior. Distributed systems also experience different types of failures such as crash, omission, timing, response, and Byzantine failures. Proper fault-tolerant techniques improve reliability, availability, and system dependability.

- **Reliable Client Server Communication**

Nov_Dec_2023

Q4) a) What are the requirements of dependable systems with respect to fault tolerance? How RPC handles the communication failure in the presence of : i) The client is unable to locate the server. ii) The request message from the client to the server is lost.

Ans. Requirements of Dependable Systems with Respect to Fault Tolerance

Dependable System

A **dependable system** is a system that users can trust to provide correct and continuous service even when faults occur.

Dependability is achieved through:

- fault tolerance,
- reliability,
- availability,
- safety,
- and maintainability.

Fault tolerance is one of the most important requirements of dependable distributed systems.

Requirements of Dependable Systems

1. Availability

Definition

The system should remain operational and accessible whenever users need it.

Even if some components fail, services should continue.

Example

Cloud services should remain available even when a server crashes.

2. Reliability

Definition

The system should perform correctly for a specified period without failure.

Example

Online banking transactions should complete correctly without errors.

3. Safety

Definition

The system should avoid catastrophic consequences even during failures.

Example

Air traffic control systems must prevent dangerous situations.

4. Maintainability

Definition

Faults should be easy to detect, repair, and recover from.

Example

Failed servers should be replaceable without stopping the entire system.

5. Integrity

Definition

Data should remain correct and protected from corruption.

Example

Database records should not become inconsistent during failures.

6. Fault Tolerance

Definition

The system should continue functioning even if some components fail.

This is achieved using:

- replication,
- redundancy,

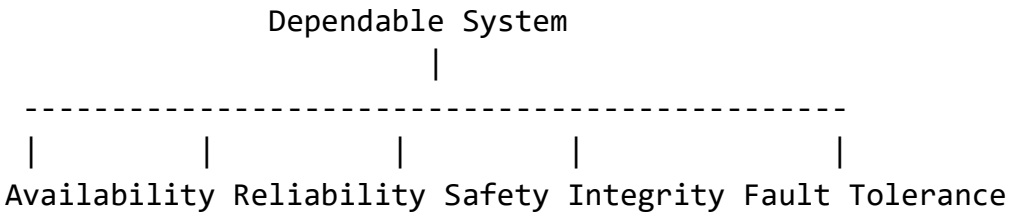
- recovery mechanisms,
- checkpointing.

7. Security

Definition

The system should protect against unauthorized access and malicious attacks.

Diagram of Dependable System Requirements



Remote Procedure Call (RPC)

Definition

RPC allows a program on one machine to invoke a procedure on another machine as if it were a local procedure call.

RPC communication involves:

- client,
- client stub,
- network,
- server stub,
- server.

Because communication occurs over a network, failures may occur.

RPC mechanisms provide methods to handle communication failures.

i) Client is Unable to Locate the Server

Problem

The client cannot find the server because:

- server address is unknown,
- server is down,

- naming service failed,
- network partition occurred.

RPC Handling Mechanism

Step 1: Binding with Name Server

The client first contacts a:

- directory service,
- binder,
- or name server

to locate the server.

Step 2: Failure Detection

If server location fails:

- timeout occurs,
- error message generated.

Step 3: Retry Mechanism

RPC may:

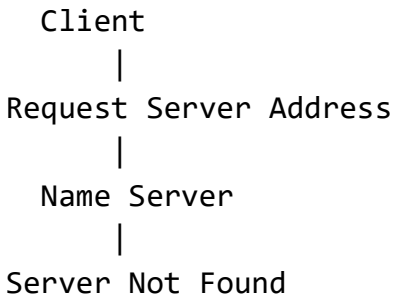
- retry locating the server,
- search alternative replicas,
- contact backup servers.

Step 4: Exception Handling

If server still cannot be found:

- RPC returns failure exception to client application.

Diagram



```
        |
Error / Retry
```

Example

A client application tries to access a distributed database server, but the naming service cannot locate the server because it has crashed.

RPC:

- retries lookup,
- or redirects to replica server.

ii) Request Message from Client to Server is Lost

Problem

The request message sent by client does not reach the server because of:

- network failure,
- packet loss,
- congestion,
- communication errors.

RPC Handling Mechanism

Step 1: Timer Starts

After sending request, client starts a timeout timer.

Step 2: No Reply Received

If server reply is not received before timeout:

- client assumes request or reply was lost.

Step 3: Retransmission

Client retransmits request message.

Step 4: Duplicate Request Handling

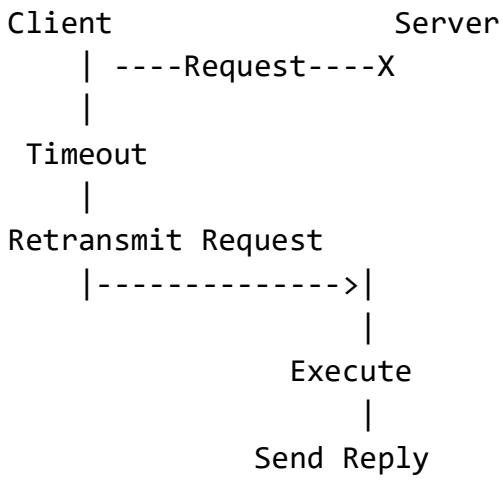
To avoid executing operation multiple times:

- sequence numbers,
- message identifiers,

- duplicate filtering

are used.

Diagram



RPC Semantics Used

RPC systems use different execution guarantees:

Semantics	Meaning
Maybe	Request may or may not execute
At-least-once	Request executed one or more times
At-most-once	Request executed at most once

Example

Suppose client requests:

`Transfer ₹500`

If request message is lost:

- client retransmits request.

To prevent duplicate money transfer:

- unique request IDs are used.

Advantages of RPC Failure Handling

- Improves reliability
- Provides transparency
- Enables automatic recovery
- Supports distributed communication

Limitations

- Increased communication overhead
- Timeout tuning is difficult
- Duplicate request management required

Applications

RPC communication is widely used in:

- distributed databases,
- cloud systems,
- microservices,
- distributed operating systems.

Technologies from Google and Microsoft heavily rely on RPC mechanisms.

Conclusion

Dependable systems require availability, reliability, safety, integrity, security, and fault tolerance to continue operating correctly during failures. RPC handles communication failures using mechanisms such as retries, timeout detection, retransmissions, duplicate filtering, and server location services. These mechanisms help distributed systems remain reliable and fault tolerant.

➢ **Reliable Group Communication**

Reliable Group Communication

Definition

Reliable Group Communication is a communication mechanism in distributed systems where a message sent to a group of processes is delivered:

- correctly,
- reliably,
- and in a consistent manner

to all intended group members, even in the presence of failures.

It ensures:

- fault tolerance,
- consistency,
- synchronization,
- and coordinated operation among distributed processes.

Need for Reliable Group Communication

In distributed systems, many applications require one process to communicate simultaneously with multiple processes.

Examples:

- distributed databases,
- replicated servers,
- multiplayer games,
- video conferencing,
- distributed file systems.

Ordinary communication may fail because of:

- message loss,
- network failures,
- process crashes,
- duplicate messages.

Reliable group communication solves these problems.

Goals of Reliable Group Communication

1. Reliability

Messages should reach all intended group members.

2. Consistency

All members should observe messages in a consistent order.

3. Fault Tolerance

Communication should continue despite failures.

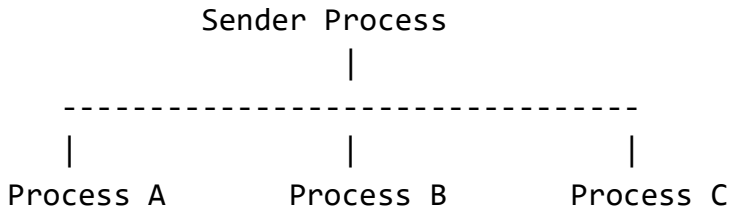
4. Scalability

System should support large groups efficiently.

Group Communication Model

A group consists of multiple processes communicating together.

Diagram



A single message is delivered to all group members.

Types of Group Communication

1. Multicast Communication

A message is sent to multiple selected processes.

2. Broadcast Communication

A message is sent to all processes in the network.

Reliability Issues in Group Communication

Problems that may occur:

- message loss,
- duplicate delivery,
- incorrect ordering,
- process crashes,
- network partitioning.

Reliable group communication protocols handle these issues.

Characteristics of Reliable Group Communication

1. Atomicity

Either:

- all group members receive the message,

- or none receive it.

This prevents partial delivery.

2. Ordering

Messages must be delivered in correct order.

Types of Ordering

FIFO Ordering

Messages from same sender arrive in order.

Causal Ordering

Causally related messages preserve order.

Total Ordering

All processes observe identical message order.

3. Reliability

Lost messages are detected and retransmitted.

4. Membership Management

Processes may:

- join groups,
- leave groups,
- or fail.

System maintains updated group membership.

Reliable Multicasting

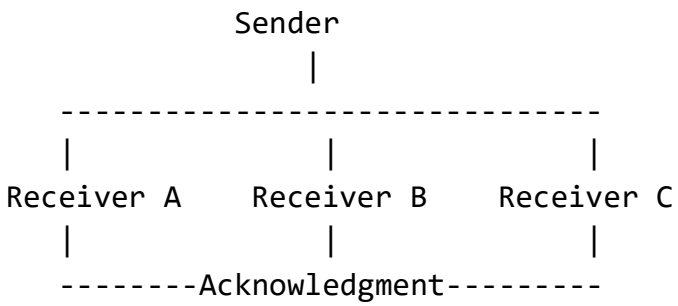
Definition

Reliable multicasting ensures that multicast messages are delivered reliably to all group members.

Working

1. Sender multicasts message.
2. Receivers send acknowledgments.
3. Missing messages are retransmitted.
4. Correct order is maintained.

Diagram



Message Ordering in Reliable Group Communication

1. FIFO Ordering

Messages from one sender are delivered in same order.

Example:

`M1 before M2`

All receivers must see:

`M1 → M2`

2. Causal Ordering

Cause-effect relationships preserved.

If:

`M1 causes M2`

then all processes see:

`M1 before M2`

3. Total Ordering

Every process receives all messages in exactly the same order.

Example

If messages are:

`M1, M2, M3`

All processes observe:

```
M1 → M2 → M3
```

Virtual Synchrony

Definition

Virtual synchrony ensures:

- all members observe group membership changes consistently,
- and receive messages in same order.

Example

If a process crashes:

- all remaining members agree on:
    - which messages were delivered,
    - and which process failed.

Applications of Reliable Group Communication

Reliable group communication is used in:

- replicated databases,
- distributed transactions,
- cloud systems,
- multiplayer games,
- collaborative applications,
- stock trading systems.

Companies like Google and Amazon use reliable group communication in distributed cloud services.

Advantages

- High reliability
- Fault tolerance
- Consistent communication
- Simplified distributed coordination
- Better scalability

Disadvantages

- Communication overhead

- Complex ordering mechanisms
- Increased synchronization cost
- Difficult implementation

Comparison: Ordinary vs Reliable Group Communication

Feature	Ordinary Communication	Reliable Group Communication
Message Delivery	Not guaranteed	Guaranteed
Fault Tolerance	Low	High
Ordering	May differ	Controlled ordering
Duplicate Handling	Limited	Managed properly
Consistency	Weak	Strong

Conclusion

Reliable group communication is an essential mechanism in distributed systems that ensures messages are delivered reliably and consistently to all group members despite failures. By supporting atomicity, ordering, reliability, and fault tolerance, it enables correct operation of distributed applications such as replicated databases, cloud services, and collaborative systems.

➢ **Distributed Commit**

Nov_Dec_2023

Q4) b) What is the distribution commit problem? Discuss how this problem is solved using the two-phase commit protocol with suitable diagram.

Ans. Distributed Commit Problem

Definition

The **distributed commit problem** occurs in distributed systems when a transaction executes across multiple sites or databases and the system must ensure that:

- either **all participating sites commit the transaction**,
- or **all sites abort the transaction**.

The system must maintain:

- consistency,
- atomicity,

- reliability,
- and fault tolerance.

Need for Distributed Commit

In distributed databases, a transaction may involve:

- multiple servers,
- multiple databases,
- or geographically distributed sites.

If one site commits while another aborts:

- data inconsistency occurs.

Therefore, all sites must reach a common agreement.

Example

Consider a bank transfer:

```
₹5000 transferred from Account A to Account B
```

- Account A database is on Server X.
- Account B database is on Server Y.

If:

- Server X deducts money,
- but Server Y fails before deposit,

then money is lost.

This creates the distributed commit problem.

Goal of Distributed Commit Protocol

The protocol must ensure:

```
Either all commit OR all abort
```

This property is called:

Atomic Commitment

Two-Phase Commit Protocol (2PC)

Definition

The **Two-Phase Commit Protocol (2PC)** is a distributed algorithm used to solve the distributed commit problem.

It coordinates all participating sites to either:

- commit the transaction together,
- or abort together.

Components of 2PC

1. Coordinator

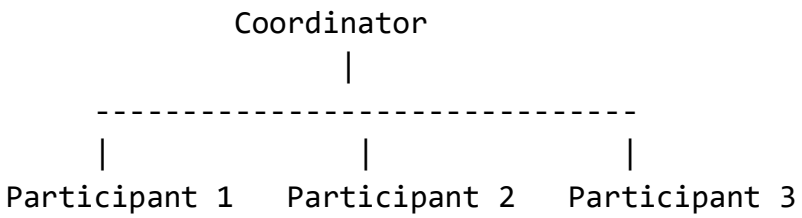
Central process responsible for:

- managing commit process,
- collecting votes,
- making final decision.

2. Participants (Cohorts)

Distributed sites involved in transaction execution.

Diagram of Two-Phase Commit Protocol



Working of Two-Phase Commit Protocol

2PC works in two phases:

1. Voting Phase (Prepare Phase)
2. Commit Phase (Decision Phase)

Phase 1: Voting / Prepare Phase

Step 1: Coordinator Sends Prepare Message

Coordinator asks all participants:

`Can you commit?`

Step 2: Participants Execute Transaction

Each participant:

- performs local transaction,
- stores updates temporarily,
- checks whether commit is possible.

Step 3: Participants Send Vote

Participants respond:

Vote-Commit

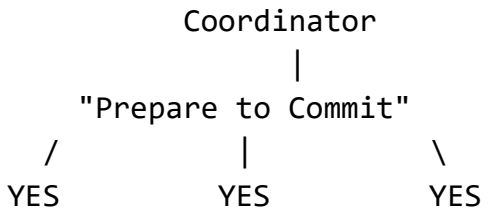
`YES`

or

Vote-Abort

`NO`

Diagram: Voting Phase



Phase 2: Commit / Abort Phase

Case 1: All Participants Vote YES

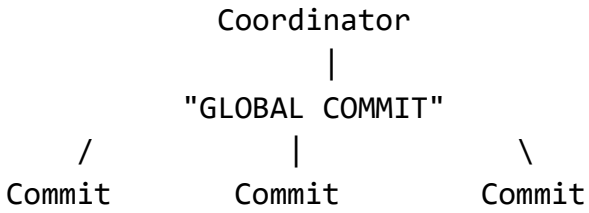
Coordinator decides:

`GLOBAL COMMIT`

Coordinator sends commit message to all participants.

Participants permanently save changes.

Diagram: Commit Phase



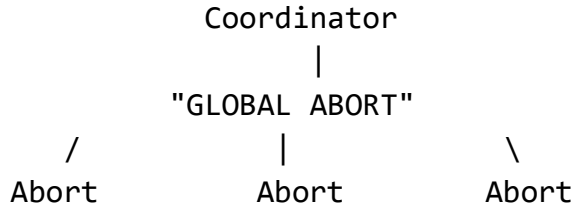
Case 2: Any Participant Votes NO

Coordinator decides:

`GLOBAL ABORT`

All participants rollback changes.

Diagram: Abort Phase



Example of Two-Phase Commit

Suppose:

- Bank Server A deducts money.
- Bank Server B credits money.

Step 1

Coordinator sends:

`Prepare`

Step 2

Both servers verify transaction feasibility.

Step 3

If both reply:

`YES`

Coordinator sends:

`COMMIT`

Both databases finalize updates.

If One Server Fails

If one participant sends:

`NO`

Coordinator sends:

`ABORT`

Both sites rollback transaction.

Characteristics of 2PC

Feature	Description
Atomicity	All commit or all abort
Consistency	Distributed data remains consistent
Fault Coordination	Handles partial failures
Central Coordination	Coordinator controls protocol

Advantages of Two-Phase Commit

- Ensures atomic transactions
- Maintains consistency
- Reliable distributed coordination
- Simple and widely used

Disadvantages of Two-Phase Commit

1. Blocking Problem

Participants may wait indefinitely if coordinator crashes.

2. Coordinator Bottleneck

Single coordinator controls entire protocol.

3. High Communication Overhead

Multiple message exchanges required.

4. Slow Performance

Waiting for all participants increases delay.

Applications

2PC is used in:

- distributed databases,
- banking systems,
- transaction processing,
- cloud systems.

Used in systems developed by Oracle and IBM.

Conclusion

The distributed commit problem ensures that distributed transactions either commit completely or abort completely across all participating sites. The Two-Phase Commit Protocol solves this problem using a coordinator and two phases: voting and final decision. It guarantees atomicity and consistency in distributed transactions, making it one of the most important protocols in distributed database systems.

- ➢ **Recovery**
- ❑ **Check Pointing**

May_Jun_2023

Q4) a) What is checkpointing in a distributed system? Explain the working of Coordinated checkpointing recovery mechanism.

Ans. Checkpointing in Distributed Systems

Definition

Checkpointing is a fault-tolerance technique in distributed systems in which the state of a process or system is periodically saved onto stable storage.

The saved state is called a:

Checkpoint

If a failure occurs, the system can restart execution from the latest checkpoint instead of restarting from the beginning.

Purpose of Checkpointing

Checkpointing helps in:

- fault recovery,
- minimizing computation loss,
- improving reliability,
- reducing recovery time.

It is widely used in:

- distributed systems,
- databases,
- cloud computing,
- parallel processing systems.

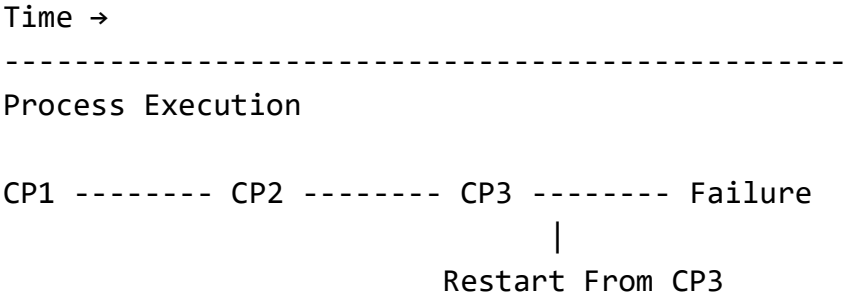
Example

Suppose a long-running scientific computation executes for 10 hours.

If the system crashes after 9 hours:

- without checkpointing → entire computation restarts,
- with checkpointing → computation resumes from latest saved checkpoint.

Diagram of Checkpointing



CP = Checkpoint

Recovery in Distributed Systems

When failures occur:

- processes must recover consistently,
- communication states must remain correct,
- and orphan or lost messages must be avoided.

Checkpointing mechanisms help achieve this.

Types of Checkpointing

1. Coordinated Checkpointing
2. Uncoordinated Checkpointing
3. Communication-Induced Checkpointing

Coordinated Checkpointing

Definition

In coordinated checkpointing, all processes in the distributed system coordinate with each other to create a consistent global checkpoint.

The checkpoints are taken simultaneously or in a synchronized manner.

This avoids:

- inconsistent states,
- orphan messages,
- domino effect.

Goal

To create a:

Consistent Global State

where:

- all saved checkpoints are mutually consistent.

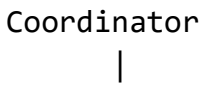
Working of Coordinated Checkpointing Recovery Mechanism

Step 1: Initiation of Checkpoint

One process (called initiator or coordinator) starts checkpointing.

It sends checkpoint request messages to all participating processes.

Diagram



Step 2: Processes Suspend Communication

Each process:

- temporarily pauses communication,
- completes ongoing operations,
- prepares to save state.

This prevents inconsistent message recording.

Step 3: Local Checkpoint Creation

Each process saves:

- process state,
- variables,
- program counter,
- communication state

to stable storage.

Diagram

```
Process P1 → Checkpoint CP1
Process P2 → Checkpoint CP2
Process P3 → Checkpoint CP3
```

Step 4: Acknowledgment to Coordinator

After checkpoint completion:

- processes send acknowledgment to coordinator.

Step 5: Resume Normal Execution

Once all checkpoints are safely stored:

- coordinator informs processes,
- normal execution resumes.

Consistent Global Checkpoint

The collection:

`(CP1, CP2, CP3)`

forms a globally consistent checkpoint.

Failure Recovery Using Coordinated Checkpointing

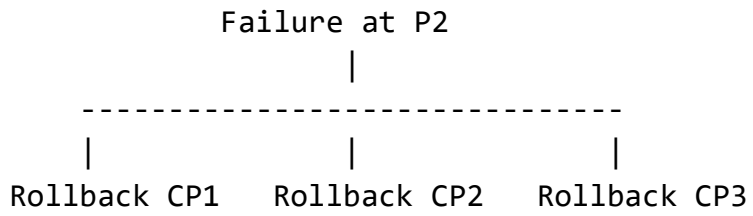
Suppose:

- Process P2 crashes.

Recovery procedure:

1. All processes rollback to latest coordinated checkpoint.
2. System restores saved states.
3. Execution resumes consistently.

Diagram of Recovery



All processes recover together.

Why Coordination is Important

Without coordination:

- one process may rollback further than others,
- inconsistent states may occur,
- messages may become orphaned.

Coordinated checkpointing prevents this.

Domino Effect

Definition

The domino effect occurs when rollback of one process forces repeated rollback of other processes to maintain consistency.

This may eventually force rollback to the beginning of computation.

Coordinated Checkpointing Avoids Domino Effect

Because all checkpoints are globally consistent.

Advantages of Coordinated Checkpointing

1. Simple Recovery

Easy rollback mechanism.

2. No Domino Effect

Consistent checkpoints prevent cascading rollbacks.

3. Consistent Global State

All processes recover uniformly.

4. Easier Implementation

Simpler dependency management.

Disadvantages

1. Synchronization Overhead

Processes must coordinate checkpoint creation.

2. Temporary Blocking

Processes may pause communication.

3. Performance Overhead

Checkpoint storage consumes resources.

Applications

Coordinated checkpointing is used in:

- distributed databases,
- cloud systems,
- parallel computing,
- scientific simulations,
- distributed transaction systems.

Used in large-scale systems developed by IBM and Oracle.

Comparison: Coordinated vs Uncoordinated Checkpointing

Feature	Coordinated	Uncoordinated
Coordination Required	Yes	No
Domino Effect	Avoided	Possible
Recovery Complexity	Simple	Complex
Consistency	High	Difficult to maintain
Overhead	Higher synchronization	Lower synchronization

Conclusion

Checkpointing is a fault-tolerance mechanism that periodically saves process states for recovery after failures. In coordinated checkpointing, all distributed processes synchronize to create a consistent global checkpoint. During recovery, processes rollback to this checkpoint, ensuring consistency and avoiding the domino effect. This mechanism improves reliability and fault recovery in distributed systems.

❑ Message Logging

Recovery – Message Logging in Distributed Systems

Definition

Message logging is a fault recovery technique in distributed systems where all messages exchanged between processes are recorded in stable storage.

If a process fails:

- the process is restarted from its latest checkpoint,
- and logged messages are replayed

to reconstruct the process state before failure.

Purpose of Message Logging

Message logging helps:

- recover lost process state,
- avoid restarting the entire system,
- reduce rollback overhead,

- improve fault tolerance.

It is commonly used with:

- checkpointing mechanisms.

Basic Idea

A process state changes because of:

1. Internal computation
2. Received messages

If all received messages are logged, the system can:

- replay those messages after failure,
- and rebuild the process state.

Components of Message Logging Recovery

1. Checkpoint

Periodic saved state of process.

2. Message Log

Record of all received messages stored on stable storage.

3. Recovery Mechanism

Uses:

- latest checkpoint,
- and replayed messages

to restore process state.

Diagram of Message Logging

```
Time →
-----------------------------------------------------

Checkpoint ---- M1 ---- M2 ---- M3 ---- Failure
                    (Messages Logged)
Recovery:
Restart from Checkpoint + Replay M1, M2, M3
```

Working of Message Logging Recovery

Step 1: Process Takes Checkpoint

The process periodically saves its state.

Step 2: Incoming Messages are Logged

Whenever a process receives a message:

- message is stored in stable storage,
- then processed normally.

Step 3: Failure Occurs

Suppose process crashes after processing several messages.

Step 4: Process Restarts

The failed process reloads:

- latest checkpoint.

Step 5: Replay Logged Messages

System replays logged messages in original order.

This reconstructs process state exactly as before failure.

Example

Suppose Process P takes checkpoint:

`CP1`

After CP1, process receives:

`M1, M2, M3`

All messages are logged.

Then process crashes.

Recovery

1. Restart from CP1
2. Replay:

```
M1 → M2 → M3
```

Process reaches pre-failure state.

Diagram of Recovery

```
            Before Failure

Checkpoint → M1 → M2 → M3 → Crash

             Recovery

Restart from Checkpoint
        ↓
Replay M1, M2, M3
        ↓
Original State Restored
```

Deterministic and Non-Deterministic Events

Deterministic Events

Events whose results are predictable.

Example:

```
x = x + 1
```

Non-Deterministic Events

Events whose outcomes depend on external inputs.

Example:

- incoming messages,
- user input,
- network packets.

These events must be logged for recovery.

Types of Message Logging

1. Pessimistic Logging
2. Optimistic Logging

3. Causal Logging

1. Pessimistic Logging

Definition

Messages are logged to stable storage before being processed.

Characteristics

- Strong recovery guarantee
- No orphan processes
- Slower performance due to disk writes

Diagram

```
Receive Message
      ↓
Log to Stable Storage
      ↓
Process Message
```

Advantages

- Simple recovery
- Consistent state guaranteed

Disadvantages

- High overhead
- Slower execution

2. Optimistic Logging

Definition

Messages are first stored temporarily and later written to stable storage asynchronously.

Characteristics

- Better performance
- Faster execution
- Risk of message loss during crash

Advantages

- Reduced overhead
- Better runtime performance

Disadvantages

- More complex recovery
- Possible orphan processes

3. Causal Logging

Definition

Tracks causal dependencies between messages while logging.

Characteristics

- Avoids orphan processes
- Better balance between pessimistic and optimistic logging

Orphan Process

Definition

A process becomes orphaned when its state depends on a message that is lost after recovery.

Example

Process P sends message M to Q.

Q processes M and changes state.

If P crashes and loses record of sending M:

- Q becomes orphan process.

Advantages of Message Logging

- Efficient recovery
- Reduced rollback
- Localized fault recovery
- Improved fault tolerance

Disadvantages

- Storage overhead
- Logging overhead
- Complex recovery management
- Dependency tracking required

Applications

Message logging is used in:

- distributed databases,
- banking systems,
- cloud computing,
- transaction systems,
- distributed file systems.

Large-scale systems by IBM and Oracle use logging-based recovery mechanisms.

Comparison of Logging Techniques

Feature	Pessimistic	Optimistic	Causal
Logging Time	Before processing	After processing	Dependency-based
Recovery Complexity	Simple	Complex	Moderate
Performance	Slower	Faster	Balanced
Orphan Processes	Avoided	Possible	Avoided

Conclusion

Message logging is an important recovery mechanism in distributed systems where received messages are recorded to stable storage. After a failure, the process restarts from the latest checkpoint and replays logged messages to recover its state. Techniques such as pessimistic, optimistic, and causal logging provide different trade-offs between performance and recovery complexity.

♦ **Case Study: Caching And Replication in Web**

Case Study: Caching and Replication in the Web

Introduction

Modern web systems serve millions of users worldwide.

To provide:

- fast access,
- high availability,
- scalability,
- and reduced network load,

web systems extensively use:

1. **Caching**
2. **Replication**

These techniques improve web performance and user experience.

Web Caching

Definition

Caching is the process of storing frequently accessed web content temporarily at locations closer to users.

Cached content may include:

- web pages,
- images,
- videos,
- scripts,
- API responses.

Goal of Web Caching

- Reduce access latency
- Minimize bandwidth usage
- Reduce server load
- Improve response time

Types of Web Caching

1. Browser Cache (Client Cache)

Web browsers locally store recently accessed content.

Example

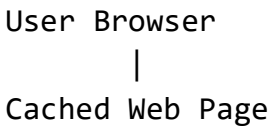
A browser stores:

- images,
- CSS files,
- JavaScript files

to avoid downloading them repeatedly.

Browsers from Google and Mozilla use browser caching extensively.

Diagram



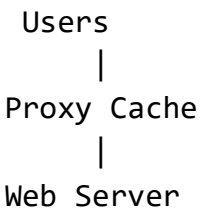
2. Proxy Cache

A proxy server stores copies of web content for multiple users.

Working

1. User requests webpage.
2. Proxy checks local cache.
3. If content exists:
    a. return cached copy.
4. Otherwise:
    a. fetch from web server,
    b. store locally,
    c. send to user.

Diagram



Advantages

- Reduced bandwidth
- Faster access
- Reduced server traffic

3. CDN (Content Delivery Network) Cache

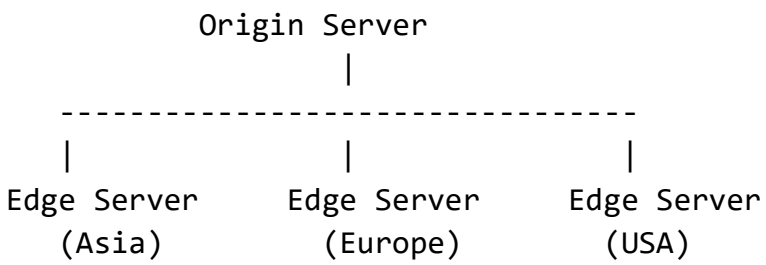
CDNs store content at geographically distributed edge servers.

Example

CDN providers:

- Cloudflare
- Akamai Technologies

Diagram



Users access nearest edge server.

Web Replication

Definition

Replication means maintaining multiple copies of web content or services on multiple servers.

Purpose of Replication

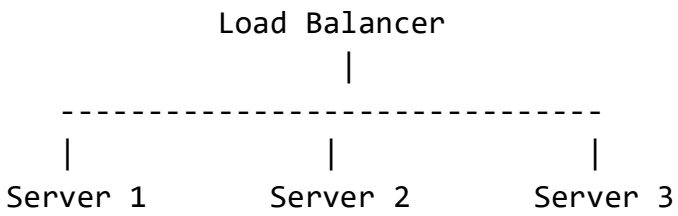
- High availability
- Fault tolerance
- Load balancing
- Scalability
- Faster access

Types of Replication in Web Systems

1. Server Replication

Multiple web servers host identical content.

Diagram



Example

Large websites use replicated servers to handle millions of requests.

Platforms like Amazon and Netflix use massive web replication infrastructures.

2. Database Replication

Databases are copied across multiple servers.

Example

E-commerce systems replicate:

- product databases,
- customer information,
- order systems.

3. Geographic Replication

Replicas are distributed across regions worldwide.

Example

Users in India access nearby servers instead of U.S. servers.

Working of Caching and Replication Together

Caching and replication often work together.

Example Scenario

Suppose a user requests a video from a streaming platform.

Step 1: DNS Redirection

User request directed to nearest CDN server.

Step 2: Cache Check

Edge server checks whether content is cached.

Step 3A: Cache Hit

If content exists:

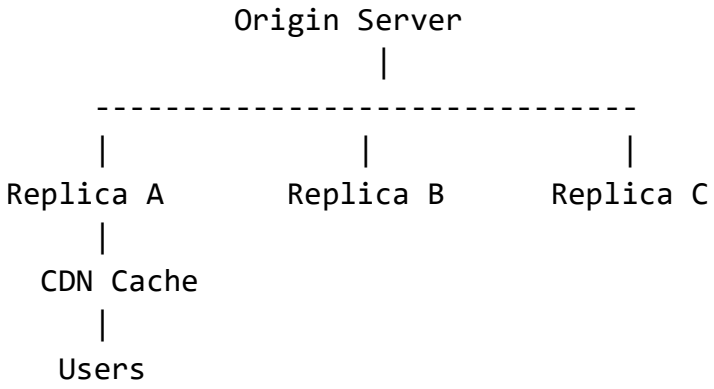
- serve immediately.

Step 3B: Cache Miss

If content absent:

- fetch from replicated origin server,
- cache locally,
- send to user.

Combined Architecture Diagram



Cache Consistency Problem

Issue

Cached content may become outdated.

Example

A webpage changes at origin server, but cache still stores old version.

Solutions

- Cache expiration
- Validation protocols

- Push updates
- Pull updates

Cache Replacement Policies

When cache becomes full, some content must be removed.

Common policies:

Policy	Description
FIFO	Remove oldest content
LRU	Remove least recently used
LFU	Remove least frequently used

Benefits of Caching and Replication in Web

Benefit	Explanation
Reduced Latency	Faster access to users
Scalability	Handles large traffic
Load Balancing	Requests distributed
Fault Tolerance	Backup servers available
Reduced Bandwidth	Cached data reused

Challenges

Challenge	Description
Cache Consistency	Keeping cache updated
Synchronization	Managing replicated data
Storage Cost	Replicas require storage
Dynamic Content	Difficult to cache frequently changing data
Security	Protecting distributed content

Real-World Example: Video Streaming

Streaming services like YouTube and Netflix use:

- replicated data centers,
- CDN caching,
- edge servers

to deliver videos globally with minimal buffering.

Conclusion

Caching and replication are fundamental techniques used in modern web systems to improve performance, scalability, reliability, and availability. Caching stores frequently accessed data closer to users, while replication maintains multiple copies of web content across servers. Together, they form the foundation of efficient large-scale web infrastructures.